Terraform Part 4

Providers, Resources & Data Sources

Professional Study Notes | Beginner to Advanced | AWS Terraform Projects | Scenario-Based Interview Preparation

Learning objectives

By the end of this part, you should be able to:

- Understand Terraform providers, provider configurations, aliases, and version constraints.
- Create and manage AWS resources using Terraform.
- Read existing infrastructure using data sources.
- Understand implicit and explicit dependencies.
- Use lifecycle rules to control resource changes and replacement.
- Apply meta-arguments such as count, for_each, depends_on, provider, and lifecycle.
- Troubleshoot real-world Terraform deployment problems and answer interview questions confidently.

Topic 1: Terraform Providers

1. Definition — What is a Provider?

A provider is a Terraform plugin that enables Terraform to communicate with an external platform or service through its APIs.

Providers allow Terraform to manage resources on platforms such as AWS, Azure, Google Cloud, Kubernetes, GitHub, and many others.

Examples of AWS provider resources include:

- aws_instance — EC2 instance
- aws_s3_bucket — S3 bucket
- aws_vpc — Virtual Private Cloud
- aws_iam_role — IAM role
- aws_eks_cluster — EKS cluster

2. Purpose — Why do we use Providers?

Providers are required because Terraform needs platform-specific functionality to create, read, update, and delete infrastructure.

Key purposes:

- Connect Terraform to cloud platforms.
- Authenticate using supported credential mechanisms.
- Configure regions and other platform-specific settings.
- Define which resource types Terraform can manage.
- Support multi-cloud and multi-provider projects.
- Control provider versions for compatibility and reliability.

3. Working — How does a Provider work?

1. Declare the provider source and version in the terraform block.
2. Configure the provider, such as the AWS region.
3. Run terraform init to initialize the working directory and install the required provider plugin.
4. Terraform uses the provider to interact with the target platform.
5. The provider communicates with the platform APIs during planning and execution.

The selected provider version is recorded in .terraform.lock.hcl, helping maintain consistent provider selections across machines.

4. Practical Example — AWS Provider Configuration

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }

  required_version = ">= 1.5.0"
}

provider "aws" {
  region = "us-east-1"
}
```

The version constraint shown permits AWS provider versions from 6.0.0 up to, but not including, 7.0.0.

Initialize Terraform:

```
terraform init
terraform providers
```

Important: Configure AWS authentication separately using an IAM role, environment variables, an AWS shared credentials profile, or another supported credential mechanism. Do not hardcode access keys in Terraform files.

5. Scenario-Based Interview Questions

Q1. Your Terraform configuration is correct, but terraform init fails to download the AWS provider. What would you check?

Answer: I would check the provider source, version constraint, Terraform Registry connectivity, proxy settings, provider mirror configuration, and the detailed initialization error.

Q2. Why do we specify a provider version?

Answer: To control compatibility and reduce unexpected behavior caused by provider upgrades. The lock file records the selected provider version and checksums.

Q3. Can Terraform use AWS and Azure providers in one project?

Answer: Yes. Terraform can use multiple providers in the same configuration, subject to the requirements of each provider and the authentication and permissions available.

Topic 2: Provider Configuration and Aliases

1. Definition

Provider configuration specifies how Terraform should connect to a platform. A provider alias identifies an additional configuration of the same provider.

2. Purpose

Aliases are useful when a project must manage infrastructure across multiple AWS regions or accounts, or when different resources require different provider configurations.

3. Working

Terraform uses the default provider configuration unless another configuration is explicitly selected. An aliased provider can be assigned to a resource through the provider meta-argument or passed into a module.

4. Practical Example — Two AWS Regions

```
provider "aws" {
  region = "us-east-1"
}

provider "aws" {
  alias  = "ohio"
  region = "us-east-2"
}

resource "aws_s3_bucket" "virginia" {
  bucket = "company-demo-virginia-12345"
}

resource "aws_s3_bucket" "ohio" {
  provider = aws.ohio
  bucket   = "company-demo-ohio-12345"
}
```

The bucket names must be unique across the S3 namespace and comply with S3 naming rules.

The first resource uses the default AWS provider; the second uses the aliased provider configured for Ohio.

5. Scenario-Based Interview Questions

Q1. Your company has infrastructure in Virginia and Ohio. How would you manage both regions?

Answer: I would configure the AWS provider for each region, assign aliases to additional configurations, and explicitly select the correct provider for each regional resource.

Q2. What happens if a resource uses the wrong provider alias?

Answer: Terraform may attempt to create or manage the resource in the wrong region or account. I would verify the provider configuration and review the plan before applying it.

Q3. Can child modules automatically access every provider alias from the root module?

Answer: Not automatically in every case. Provider configurations needed by child modules should be passed explicitly using the module's providers argument when appropriate.

Topic 3: Provider Version Constraints and Lock File

1. Definition

Provider version constraints define which versions of a provider Terraform is allowed to use. The .terraform.lock.hcl file records the selected provider versions and package checksums.

2. Purpose

- Prevent unexpected provider upgrades.
- Improve consistency between developer machines and Jenkins.
- Manage compatibility with Terraform configurations.
- Make provider installation more predictable.
- Support controlled upgrades and troubleshooting.

3. Working

Terraform reads the version constraint from the required_providers block. During initialization, it selects a compatible version and records the selection in the lock file.

Common constraint examples:

Constraint	Meaning
= 6.42.0	Exactly version 6.42.0
>= 6.0.0	Version 6.0.0 or newer
~> 6.0	Version 6.x, but not 7.0 or newer
~> 6.42.0	Version 6.42.x, but not 6.43.0 or newer

The exact versions available depend on the provider registry and platform.

4. Practical Example

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

Useful commands:

```
terraform init
terraform providers
terraform init -upgrade
```

- terraform init initializes providers and dependencies.
- terraform providers shows provider requirements in the configuration and state.
- terraform init -upgrade considers newer versions permitted by the constraints and updates selections as appropriate.

Commit .terraform.lock.hcl to Git for consistent provider selection.

5. Scenario-Based Interview Questions

Q1. Terraform works locally but uses a different provider version in Jenkins. How would you fix this?

Answer: I would check whether the lock file is committed, whether Jenkins uses the same configuration and lock file, and whether the pipeline runs terraform init -upgrade. I would standardize initialization and upgrade providers only through a reviewed change.

Q2. Does the lock file replace the version constraint?

Answer: No. The constraint defines acceptable versions, while the lock file records the selected version and checksums.

Q3. When should you use terraform init -upgrade?

Answer: When intentionally updating provider or module selections within their permitted constraints. I would test the changes and review the resulting plan before production deployment.

Topic 4: Terraform Resources

1. Definition

A resource is a Terraform configuration block that declares an infrastructure object Terraform should manage.

Resources are the primary building blocks for provisioning infrastructure.

Examples:

- EC2 instances
- VPCs and subnets
- Security groups
- S3 buckets
- IAM roles and policies
- EKS clusters and node groups

2. Purpose

Resources are used to:

- Create new infrastructure.
- Update supported attributes.
- Replace resources when required.
- Delete infrastructure no longer required by the configuration.
- Maintain infrastructure through repeatable code.
- Integrate infrastructure management into CI/CD.

3. Working

The general resource syntax is:

```
resource "RESOURCE_TYPE" "LOCAL_NAME" {
  # Resource arguments
}
```

For example:

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = "t3.micro"
}
```

Here:

- resource declares a managed resource.

- aws_instance is the resource type.
- web is the local Terraform name.
- ami and instance_type are resource arguments.

Terraform identifies this resource by its address:

```
aws_instance.web
```

If it is inside a module, the address could be:

```
module.ec2.aws_instance.web
```

4. Practical Example — Create an EC2 Instance

```
variable "ami_id" {
  description = "Valid AMI ID for the selected AWS region"
  type        = string
}

resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  tags = {
    Name        = "terraform-web-server"
    Environment = "dev"
    ManagedBy   = "Terraform"
  }
}
```

The AMI ID must exist in the configured AWS region, and the executing identity must have sufficient permissions.

Run:

```
terraform fmt
terraform validate
terraform plan
terraform apply
```

Terraform creates the EC2 instance if the plan and required permissions permit it.

5. Scenario-Based Interview Questions

Q1. Your team needs 10 EC2 instances with similar configurations. Would you write 10 resource blocks?

Answer: Not necessarily. I would consider count or for_each, depending on whether the instances are identical or have distinct identities and configurations.

Q2. You change an instance type from t3.micro to t3.small. Will Terraform always replace the instance?

Answer: Not necessarily. The AWS provider may support changing the instance type in place. I would inspect the actual plan because behavior depends on the resource and the attribute changed.

Q3. What happens when you remove a resource block from the configuration?

Answer: If the resource is managed by the current state, Terraform normally proposes destroying it. I would review the plan before applying, especially for critical resources.

Topic 5: Resource Arguments and Attributes

1. Definition

Arguments are configuration values supplied to a resource. Attributes are values exposed by a resource or data source, including computed values returned by the provider.

2. Purpose

Arguments configure infrastructure, while attributes let other resources and outputs use information returned by the provider.

3. Working

An argument may be required or optional. An attribute may be known during planning or only after the resource has been created.

For example, you can provide an AMI ID as an argument and reference the instance ID as an attribute.

4. Practical Example

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  tags = {
    Name = "web-server"
  }
}

output "instance_id" {
  value = aws_instance.web.id
}

output "public_ip" {
  value = aws_instance.web.public_ip
}
```

Here, ami and instance_type are arguments. The id and public_ip attributes are provided by the resource.

The public IP may be unknown until the instance is created, and it may be absent if the instance has no public IP address.

5. Scenario-Based Interview Questions

Q1. What is the difference between an argument and an attribute?

Answer: An argument is a configuration value supplied to a block. An attribute is a value exposed by a resource or data source.

Q2. Why does Terraform show (known after apply)?

Answer: Terraform cannot determine the value during planning, often because it is generated by the provider or the cloud platform during creation.

Q3. Can one resource use another resource's attribute?

Answer: Yes. Referencing an attribute such as aws_security_group.web.id lets Terraform use that value and normally creates an implicit dependency.

Topic 6: Data Sources

1. Definition

A data source retrieves information about existing infrastructure or other information exposed by a provider. It reads information rather than declaring ownership of the object as a managed resource.

2. Purpose

Data sources are useful when:

- A VPC already exists.
- An existing subnet must be selected.
- The latest suitable AMI must be looked up.
- An existing security group or IAM resource must be referenced.
- Infrastructure is shared between teams or Terraform configurations.

3. Working

The syntax is:

```
data "DATA_SOURCE_TYPE" "LOCAL_NAME" {
  # Lookup arguments or filters
}
```

Terraform asks the provider to retrieve matching information. Other resources can reference the returned attributes.

A data source can fail if no matching object exists or the caller lacks permission to read it.

4. Practical Example — Look Up an Existing VPC

```
data "aws_vpc" "existing" {
  id = var.vpc_id
}

output "existing_vpc_id" {
  value = data.aws_vpc.existing.id
}
```

The vpc_id variable must contain an existing VPC ID accessible through the configured AWS provider.

Use this data source when you want to retrieve a known VPC rather than create a new one.

Another common example is selecting a subnet:

```
data "aws_subnet" "selected" {
  id = var.subnet_id
}

output "subnet_cidr" {
  value = data.aws_subnet.selected.cidr_block
}
```

5. Scenario-Based Interview Questions

Q1. Your organization already has a VPC managed by another team. How would you use it in your Terraform project?

Answer: I would retrieve the VPC through an appropriate data source, reference its ID and relevant subnet IDs, and create only the resources my project is responsible for managing.

Q2. Does a data source create a new resource?

Answer: No. It reads information about an existing object or retrieves provider information. Resource blocks are used to manage infrastructure objects.

Q3. Your VPC data source fails with a not-found error. What would you check?

Answer: I would verify the VPC ID, AWS account, provider region, resource existence, and the permissions available to the Terraform identity.

Topic 7: Resource vs. Data Source

1. Definition

A resource manages an infrastructure object, while a data source retrieves information about an object or provider-managed data.

2. Purpose

The distinction prevents teams from accidentally creating duplicate infrastructure or attempting to manage resources that belong to another team.

3. Working

Feature	Resource	Data source
Syntax	resource	data
Main purpose	Manage an object	Read information
Can create an object?	Yes, when supported	No
Can update the object?	Yes, when supported	Does not manage its lifecycle
Can delete the object?	Yes, when removed from management and planned accordingly	No
Typical use	Create EC2 instance	Look up existing VPC

4. Practical Example

Create a security group:

```
resource "aws_security_group" "app" {
  name = "app-security-group"
}
```

Read an existing VPC:

```
data "aws_vpc" "existing" {
  id = var.vpc_id
}
```

The security group is managed by Terraform. The VPC is looked up and is not created or destroyed by this data source.

5. Scenario-Based Interview Questions

Q1. You need an EC2 instance in an existing VPC. Which blocks would you use?

Answer: I would use a data source to retrieve the existing VPC and relevant subnet information, then use an EC2 resource to create the instance.

Q2. What happens if you delete a data source block?

Answer: Terraform stops querying that data source as part of the configuration. It does not destroy the underlying object.

Q3. Can Terraform import an existing resource?

Answer: Yes. A supported existing resource can be imported into Terraform state and then managed through a matching resource configuration. A data source is not a substitute for importing an object that you intend to manage.

Topic 8: Implicit Dependencies

1. Definition

An implicit dependency is a dependency Terraform automatically identifies when one resource references another resource's attribute.

2. Purpose

Implicit dependencies allow Terraform to determine a suitable execution order without requiring engineers to specify every dependency manually.

3. Working

When Terraform sees a reference to another resource, it builds a dependency relationship.

For example:

```
resource "aws_security_group" "app" {
  name = "app-security-group"
}

resource "aws_instance" "app" {
  ami                    = var.ami_id
  instance_type          = "t3.micro"
  vpc_security_group_ids = [aws_security_group.app.id]
}
```

The EC2 instance references the security group ID. Terraform therefore knows the security group must be created before the instance can use it.

4. Practical Example

In an AWS application project, you might create:

1. A security group.
2. An EC2 instance that references the security group.
3. An output that references the instance ID.

Terraform infers the relevant dependencies from these references.

5. Scenario-Based Interview Questions

Q1. Why does Terraform create a security group before an EC2 instance?

Answer: The instance references the security group ID, so Terraform infers a dependency and orders the operations accordingly.

Q2. Do you need depends_on every time one resource uses another resource's ID?

Answer: No. A normal resource reference generally creates the necessary implicit dependency.

Q3. Can Terraform create independent resources in parallel?

Answer: Yes. Resources with no dependency relationship may be processed concurrently, subject to Terraform's parallelism settings and provider behavior.

Topic 9: Explicit Dependencies — depends_on

1. Definition

depends_on is a meta-argument that explicitly tells Terraform a resource or module must depend on another resource or module.

2. Purpose

It is useful when an operational dependency exists but cannot be reliably inferred from a direct reference.

For example, an application may need an IAM policy attachment to be completed before an application resource is created, even if that application resource does not directly reference the attachment.

3. Working

Terraform creates a dependency edge in its dependency graph. During creation, the dependency is handled first; during destruction, the dependent resource is generally handled before its dependency.

Use explicit dependencies only when necessary. They can make plans more conservative and increase the number of values that remain unknown until apply.

4. Practical Example

```
resource "aws_iam_role" "app" {
  name = "application-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Service = "ec2.amazonaws.com"
      }
      Action = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "app" {
  role       = aws_iam_role.app.name
  policy_arn = var.application_policy_arn
}

resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  depends_on = [
    aws_iam_role_policy_attachment.app
  ]
}
```

This example demonstrates an explicit ordering dependency. In a real application, ensure that the EC2 instance is actually configured to use the role through an appropriate instance profile; an IAM role attachment alone does not attach the role to the instance.

5. Scenario-Based Interview Questions

Q1. When would you use depends_on instead of an ordinary resource reference?

Answer: When Terraform must respect a dependency that is not expressed through a normal reference, such as an operational requirement that a policy attachment exist before another resource is provisioned.

Q2. What is the disadvantage of unnecessary depends_on?

Answer: It can restrict parallel execution, cause more values to remain unknown during planning, and make the dependency graph harder to maintain.

Q3. Can you use depends_on with a module?

Answer: Yes. A module block can depend explicitly on resources or modules when a genuine dependency cannot be inferred from normal references.

Topic 10: Terraform Lifecycle Rules

1. Definition

The lifecycle meta-argument controls certain aspects of how Terraform manages changes to a resource.

Important lifecycle rules include:

- create_before_destroy
- prevent_destroy
- ignore_changes
- replace_triggered_by

2. Purpose

Lifecycle rules help control resource replacement, protect critical infrastructure from certain planned deletions, and manage attributes that are changed outside Terraform intentionally.

3. Working

Terraform evaluates lifecycle settings when building a plan and determining how resources should be changed.

These settings influence Terraform's behavior, but they do not replace cloud permissions, backups, change approvals, or operational safeguards.

4. Practical Example

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  lifecycle {
    prevent_destroy = true
  }
}
```

This configuration instructs Terraform to reject a plan that would destroy this resource while the rule remains applicable in the configuration.

Important: prevent_destroy does not protect against every possible deletion. For example, removing the entire resource block also removes the lifecycle rule from the configuration. It does not prevent deletion directly through the AWS Console or API.

5. Scenario-Based Interview Questions

Q1. How would you protect a critical resource against accidental Terraform destruction?

Answer: I would consider prevent_destroy alongside least-privilege IAM, production approval, plan reviews, backups, and monitoring. I would not rely on one lifecycle rule alone.

Q2. Does prevent_destroy stop an administrator from deleting the resource manually in AWS?

Answer: No. It influences Terraform's planning behavior, not the cloud platform's independent deletion controls.

Q3. What happens if Terraform must replace a protected resource?

Answer: A replacement requires destruction of the existing object, so prevent_destroy will generally cause Terraform to reject the destructive plan. The protection must be reviewed and changed through an approved process if replacement is genuinely required.

Topic 11: create_before_destroy

1. Definition

create_before_destroy is a lifecycle rule that instructs Terraform to create a replacement resource before destroying the existing resource when replacement is required and the provider and platform allow this sequence.

2. Purpose

It can reduce downtime during resource replacement by ensuring the new resource is created before the old one is removed.

3. Working

Without this rule, Terraform commonly destroys the existing object before creating its replacement when the resource requires replacement.

With create_before_destroy = true, Terraform attempts the reverse order. However, the old and new objects must be able to coexist. Unique-name restrictions, address conflicts, quotas, and dependencies can prevent the strategy from working.

4. Practical Example

```
resource "aws_iam_role" "app" {
  name = "app-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Service = "ec2.amazonaws.com"
      }
      Action = "sts:AssumeRole"
    }]
  })

  lifecycle {
    create_before_destroy = true
  }
}
```

This illustrates the lifecycle setting, but the example's fixed role name may prevent simultaneous creation of a replacement with the same name. A production design may need versioned names or another replacement strategy.

5. Scenario-Based Interview Questions

Q1. A production resource must be replaced with minimal downtime. What would you consider?

Answer: I would assess whether create_before_destroy is supported and whether both versions can coexist. For services requiring continuous availability, I might also use blue-green deployment, load balancing, health checks, and a controlled traffic switch.

Q2. Why can create_before_destroy fail?

Answer: The replacement may require the same unique name, exceed quotas, or conflict with other infrastructure dependencies.

Q3. Does create_before_destroy guarantee zero downtime?

Answer: No. It changes replacement ordering, but zero downtime depends on architecture, health checks, traffic routing, compatibility, and application behavior.

Topic 12: ignore_changes

1. Definition

ignore_changes is a lifecycle rule that tells Terraform to disregard changes to specified resource attributes when evaluating updates after the resource has been created.

2. Purpose

It is useful when certain attributes are intentionally managed by another system, such as an autoscaler or an operational automation process.

3. Working

Terraform continues managing the resource, but it does not attempt to reconcile the listed attributes in the usual way when they drift from configuration.

This should be used selectively because it can hide meaningful changes.

4. Practical Example — ECS Desired Count

```
resource "aws_ecs_service" "app" {
  name            = "application-service"
  cluster         = aws_ecs_cluster.app.id
  task_definition = aws_ecs_task_definition.app.arn
  desired_count   = 2

  lifecycle {
    ignore_changes = [desired_count]
  }
}
```

This illustrates a service whose desired task count may be managed by Application Auto Scaling. After creation, Terraform ignores changes to desired_count during normal update planning.

The other service attributes remain managed. The referenced ECS resources and their configurations must also be defined correctly.

5. Scenario-Based Interview Questions

Q1. An autoscaler changes the number of running application instances. Terraform wants to revert the change. What would you consider?

Answer: If autoscaling is intentionally responsible for that attribute, I would consider ignore_changes for the appropriate attribute. I would also ensure that ownership between Terraform and the autoscaler is clearly defined.

Q2. Should you use ignore_changes = all on every resource?

Answer: No. That would hide many configuration changes and weaken Terraform's ability to detect and correct drift.

Q3. Does ignore_changes prevent someone from changing infrastructure manually?

Answer: No. It affects Terraform's handling of specified attributes; it does not prevent manual changes or enforce cloud permissions.

Topic 13: replace_triggered_by

1. Definition

replace_triggered_by is a lifecycle rule that causes a resource to be replaced when specified managed resources or their relevant attributes change in a way that triggers replacement.

2. Purpose

It is useful when a resource must be recreated in response to a related resource change, even when its own configuration does not otherwise require replacement.

3. Working

Terraform evaluates the referenced resource changes and plans replacement of the dependent resource when the lifecycle condition is triggered.

4. Practical Example

```
resource "terraform_data" "app_version" {
  input = var.app_version
}

resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  lifecycle {
    replace_triggered_by = [
      terraform_data.app_version
    ]
  }
}
```

Here, a change that updates the terraform_data.app_version resource can trigger replacement of the EC2 instance.

This is an illustrative pattern. Replacing an EC2 instance for every application version change is not always appropriate; a deployment mechanism that updates the application independently may be safer and more efficient.

5. Scenario-Based Interview Questions

Q1. An application resource must be replaced whenever a related managed resource changes. What lifecycle feature could help?

Answer: I would evaluate replace_triggered_by, provided the dependency represents a real replacement requirement.

Q2. Can replace_triggered_by refer to an ordinary input variable directly?

Answer: It expects references to managed resources or their attributes rather than arbitrary variable expressions. A terraform_data resource can be used to represent a value change when appropriate.

Q3. Is replacing a server always the best response to an application version change?

Answer: No. I would assess whether a rolling or blue-green application deployment can achieve the goal without replacing the underlying server.

Topic 14: Meta-Arguments in Terraform

1. Definition

Meta-arguments are special arguments supported by Terraform constructs that control how Terraform manages resources or modules rather than defining the business-specific properties of the resource itself.

2. Purpose

They help engineers:

- Create multiple resource instances.
- Control dependencies.
- Select provider configurations.
- Configure lifecycle behavior.
- Control replacement and protection behavior.
- Reuse a single resource definition across multiple objects.

3. Working

Important meta-arguments include:

Meta-argument	Purpose
count	Create multiple indexed instances
for_each	Create instances for keys or set members
depends_on	Define explicit dependencies
provider	Select a provider configuration
lifecycle	Control lifecycle behavior
providers	Pass provider configurations into a module
provisioner	Execute commands or other actions during specific resource lifecycle events; use sparingly

count and for_each are alternatives for defining multiple instances of a given resource or module block. They should not be used together in the same block.

4. Practical Example

```
resource "aws_instance" "app" {
  count         = 3
  ami           = var.ami_id
  instance_type = "t3.micro"

  tags = {
    Name = "app-${count.index + 1}"
  }
}
```

This creates three instances with indexed resource addresses:

```
aws_instance.app[0]
aws_instance.app[1]
aws_instance.app[2]
```

5. Scenario-Based Interview Questions

Q1. How would you create multiple EC2 instances without duplicating resource blocks?

Answer: I would use count for similar instances or for_each when each instance has a distinct key and identity.

Q2. Can you use count and for_each together in one resource block?

Answer: No. I would select the meta-argument that best represents the intended resource identity and scaling behavior.

Q3. What is the difference between provider and providers?

Answer: provider selects a provider configuration for a resource. providers maps provider configurations when calling a module.

Topic 15: count Meta-Argument

1. Definition

count creates multiple instances of a resource or module using a numeric count.

2. Purpose

It is useful when you need several similar infrastructure objects, such as a fixed number of EC2 instances.

3. Working

Terraform evaluates count and creates that number of instances. Each instance is identified by a numeric index beginning at zero.

Inside a resource that uses count, count.index provides the current instance index.

4. Practical Example

```
variable "server_count" {
  type    = number
  default = 3
}

resource "aws_instance" "app" {
  count         = var.server_count
  ami           = var.ami_id
  instance_type = "t3.micro"

  tags = {
    Name = "application-${count.index + 1}"
  }
}
```

If server_count is 3, Terraform creates three EC2 instances.

Useful commands:

```
terraform plan
terraform state list
```

5. Scenario-Based Interview Questions

Q1. You reduce count from five to three. What happens?

Answer: Terraform normally plans to destroy the instances whose indexes are no longer required, such as indexes 3 and 4. I would review the plan because index-based identity can be problematic when instances have different purposes.

Q2. When is count a good choice?

Answer: When instances are largely interchangeable and index-based identities are acceptable.

Q3. What is a common risk of using count for differently named servers?

Answer: Removing an element from the middle of a list can shift index assignments and cause unintended updates or replacements. for_each is often better when each server has a stable identity.

Topic 16: for_each Meta-Argument

1. Definition

for_each creates a resource or module instance for each element in a map or set of strings.

2. Purpose

It is useful when resources have distinct names, settings, or identities and should be tracked by stable keys rather than numeric indexes.

3. Working

Terraform creates one instance per key or set member. Within the resource block, each.key refers to the key, and each.value refers to the associated value for a map.

The map or set used by for_each must be known before Terraform performs remote resource operations.

4. Practical Example

```
variable "servers" {
  type = map(string)

  default = {
    dev     = "t3.micro"
    test    = "t3.small"
    staging = "t3.medium"
  }
}

resource "aws_instance" "app" {
  for_each      = var.servers
  ami           = var.ami_id
  instance_type = each.value

  tags = {
    Name        = "app-${each.key}"
    Environment = each.key
  }
}
```

The resource addresses will look like:

```
aws_instance.app["dev"]
aws_instance.app["test"]
aws_instance.app["staging"]
```

5. Scenario-Based Interview Questions

Q1. Your company needs separate development, testing, and staging servers with different instance types. Would you use count or for_each?

Answer: I would generally use for_each because each environment has a meaningful, stable key and can have its own instance type.

Q2. Why must for_each keys be known during planning?

Answer: Terraform needs to know the instance identities to build its plan. Keys derived only from values that become available after resource creation cannot generally determine the resource instances in advance.

Q3. What happens if you remove a key from a for_each map?

Answer: Terraform normally plans to destroy the corresponding instance because that key is no longer declared.

Topic 17: provider and providers Meta-Arguments

1. Definition

The provider meta-argument selects a provider configuration for a resource. The providers meta-argument on a module block passes provider configurations from the calling module to a child module.

2. Purpose

These arguments support multi-region, multi-account, and modular infrastructure designs.

3. Working

A resource can select an aliased provider:

```
provider = aws.ohio
```

A module can receive a provider configuration:

```
module "app_ohio" {
  source = "./modules/app"

  providers = {
    aws = aws.ohio
  }
}
```

The child module must declare the required provider appropriately. Provider configurations and aliases should be designed carefully when using multiple accounts or regions.

4. Practical Example

A company manages shared networking in one AWS region and application resources in another. The root module configures the required provider aliases and passes the correct provider configuration into each child module.

This avoids accidentally creating resources in the wrong region.

5. Scenario-Based Interview Questions

Q1. How do you pass an aliased AWS provider into a child module?

Answer: I use the module block's providers map to map the child's provider name to the desired root provider configuration.

Q2. Why might a child module create resources in an unexpected region?

Answer: The wrong provider configuration may be selected or passed to the module. I would check the provider aliases, module mappings, and the execution plan.

Q3. Does defining two provider aliases automatically create two accounts?

Answer: No. Aliases are configurations, not accounts themselves. The authentication configuration and account permissions determine which account each provider accesses.

Topic 18: Dynamic Blocks

1. Definition

A dynamic block generates repeated nested configuration blocks from a collection.

Unlike count and for_each on a resource, a dynamic block generates nested blocks within a resource or another supported configuration block.

2. Purpose

It reduces repetitive nested configuration and makes resource definitions easier to maintain when the number of nested blocks varies.

3. Working

A dynamic block specifies:

- The nested block type to generate.
- A for_each collection.
- An iterator, which is optional.
- A content block defining each generated block.

4. Practical Example

```
variable "ingress_rules" {
  type = list(object({
    from_port   = number
    to_port     = number
    protocol    = string
    cidr_blocks = list(string)
  }))
}

resource "aws_security_group" "app" {
  name = "app-security-group"

  dynamic "ingress" {
    for_each = var.ingress_rules
    iterator = rule

    content {
      from_port   = rule.value.from_port
      to_port     = rule.value.to_port
      protocol    = rule.value.protocol
      cidr_blocks = rule.value.cidr_blocks
    }
  }
}
```

The ingress_rules variable supplies the rules. Each collection element generates an ingress block.

For a production security group, allow only required traffic and trusted source ranges.

5. Scenario-Based Interview Questions

Q1. A security group needs a variable number of ingress rules. How would you avoid repeating blocks?

Answer: I would use a dynamic block with a structured input variable, validate the inputs, and review the resulting security rules in the plan.

Q2. Is a dynamic block the same as a resource using for_each?

Answer: No. A resource-level for_each creates separate resource instances. A dynamic block generates repeated nested blocks within a parent configuration.

Q3. Should dynamic blocks be used everywhere?

Answer: No. I would use them when nested repetition is genuinely variable. Straightforward static configuration can be easier to read without dynamic blocks.

Topic 19: Provisioners — local-exec and remote-exec

1. Definition

Provisioners run commands or scripts during certain resource lifecycle events. Common types include:

- local-exec — Runs a command on the machine executing Terraform.
- remote-exec — Runs commands on a remote machine, typically through a connection.

2. Purpose

Provisioners can handle limited tasks that cannot reasonably be performed through a provider resource or another dedicated tool.

3. Working

Terraform executes provisioners at specific points in the resource lifecycle, depending on their configuration. Their behavior is different from declarative resource management: Terraform does not generally track the detailed effects of arbitrary commands in the same way it tracks provider-managed resource attributes.

Provisioner failures can leave partially configured infrastructure.

4. Practical Example

```
resource "terraform_data" "example" {
  provisioner "local-exec" {
    command = "echo Terraform deployment started"
  }
}
```

This demonstrates a local command. On Windows, the command shell and quoting behavior depend on the execution environment.

For configuring applications on servers, prefer cloud-init, EC2 user data, image-building tools, configuration management, or deployment pipelines when suitable.

5. Scenario-Based Interview Questions

Q1. Should you use remote-exec to install every application on an EC2 instance?

Answer: Generally no. I would prefer repeatable image builds, cloud-init or user data for suitable bootstrap tasks, configuration-management tools, or an application deployment pipeline.

Q2. What happens if a provisioner fails?

Answer: The operation may fail while leaving the resource or some side effects in place. I would inspect the logs and actual infrastructure, then use a controlled recovery process.

Q3. Why are provisioners considered a last resort?

Answer: They introduce imperative behavior, may be difficult to repeat safely, and often have limited visibility into their side effects.

Topic 20: Resource Lifecycle — Create, Update, Replace and Destroy

1. Definition

Resource lifecycle describes the operations Terraform performs to bring infrastructure into the desired state.

The major operations are:

- Create: Provision a new resource.
- Update: Change supported attributes of an existing resource.
- Replace: Destroy and recreate a resource when an attribute or other condition requires replacement.
- Destroy: Remove a managed resource.

2. Purpose

Understanding lifecycle behavior helps prevent accidental downtime, data loss, unnecessary replacements, and unexpected infrastructure costs.

3. Working

Terraform calculates the required actions using the configuration, state, provider behavior, and available information about the infrastructure.

For example, a change to an EC2 instance's tags may be applied in place, whereas changing its AMI typically requires replacement.

The exact behavior is provider- and attribute-dependent. The Terraform plan is the primary way to inspect the intended action.

4. Practical Example

```
terraform plan
```

A plan might show these symbols:

Plan symbol	Meaning
+	Create
~	Update in place
-	Destroy
-/+	Destroy and then create
+/-	Create and then destroy

Review the detailed plan rather than relying only on the symbols.

5. Scenario-Based Interview Questions

Q1. Terraform shows that an EC2 instance will be replaced. What would you do?

Answer: I would inspect which attribute requires replacement, verify whether the change is expected, assess downtime and data risks, and plan a safe deployment before applying.

Q2. Why might Terraform replace a resource instead of updating it?

Answer: Some attributes are immutable or require replacement according to the provider's resource implementation or the underlying platform.

Q3. How do you determine whether a change will cause downtime?

Answer: I review the plan, provider behavior, dependencies, and service architecture. A plan alone cannot guarantee application availability, so I also assess traffic routing, health checks, redundancy, and recovery procedures.

Topic 21: Resource Drift and Refresh

1. Definition

Configuration drift occurs when actual infrastructure differs from the desired configuration. Drift can result from manual changes, automated external systems, or changes made outside Terraform.

2. Purpose

Drift detection helps teams identify differences between the infrastructure they expect and the infrastructure that actually exists.

3. Working

Terraform refreshes information about managed resources during planning and other relevant operations. It compares the observed information with the configuration and state to determine whether changes are needed.

Modern Terraform workflows generally use terraform plan to inspect differences rather than relying on the older standalone terraform refresh workflow.

4. Practical Example

Suppose your Terraform configuration specifies:

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.micro"
}
```

An administrator manually changes the instance type in AWS to t3.small.

Run:

```
terraform plan
```

Terraform may propose changing the instance back to t3.micro, depending on the provider's behavior and the current configuration.

Before applying, determine whether the manual change was intentional. If the new size is required, update the Terraform configuration instead of repeatedly fighting the external change.

5. Scenario-Based Interview Questions

Q1. An administrator manually changes a security group in AWS. How would you identify the drift?

Answer: I would run a Terraform plan and review the detected differences. I would determine whether to accept the change in code or restore the intended configuration.

Q2. Will Terraform automatically revert every manual change immediately?

Answer: No. Terraform does not continuously reconcile infrastructure by default. Changes are detected and acted on during relevant operations such as planning and applying.

Q3. What is the difference between drift and a configuration error?

Answer: Drift is a difference between actual infrastructure and the desired or recorded configuration. A configuration error is an issue in the Terraform code or its inputs; the two can occur together.

Topic 22: Importing Existing Infrastructure

1. Definition

Importing is the process of associating an existing infrastructure object with a Terraform resource address so that Terraform can manage it.

2. Purpose

Importing is useful when:

- A resource was created manually.
- An existing resource must be brought under Terraform management.
- A team is migrating from manual provisioning to IaC.
- Resources must be incorporated into a standardized Terraform project.

3. Working

Terraform must know which resource block represents the object and which real infrastructure object should be associated with it.

The import operation associates the existing object with Terraform state. You must also ensure that the resource configuration accurately represents the intended infrastructure.

Import does not automatically generate a complete, correct configuration for every resource.

4. Practical Example

Suppose an existing AWS security group must be managed by Terraform.

```
resource "aws_security_group" "app" {
  name = "existing-app-security-group"

  # Add the remaining configuration required
  # to match the intended existing security group.
}
```

Then use the appropriate import identifier:

```
terraform import aws_security_group.app sg-0123456789abcdef0
```

The ID above is illustrative. Use the real security group ID and the import format documented for the resource.

After import:

```
terraform plan
```

Review the plan and adjust the configuration before applying changes.

5. Scenario-Based Interview Questions

Q1. Your organization has 50 manually created AWS resources. Must you recreate them with Terraform?

Answer: Not necessarily. I would identify which resources Terraform should manage, prepare the corresponding configuration, import supported existing resources, and reconcile the configuration with their actual settings.

Q2. Does importing a resource immediately modify it in AWS?

Answer: Import primarily associates the object with Terraform state. Subsequent planning may show changes needed to reconcile the configuration with the object.

Q3. What happens if you import the wrong resource ID?

Answer: The wrong object may be associated with the Terraform address, or the import may fail. I would verify the account, region, resource ID, and address before proceeding.

Topic 23: Practical Project — EC2 with Existing VPC and Security Group

This example combines providers, resources, data sources, and dependencies in a single AWS configuration.

1. Definition

The project creates an EC2 instance in an existing VPC using an existing subnet and an existing security group.

2. Purpose

This approach is useful when the networking team manages shared infrastructure and the application team is responsible only for application compute resources.

3. Working

1. Configure the AWS provider.
2. Retrieve the existing subnet and security group.
3. Reference their IDs from the EC2 resource.
4. Let Terraform infer the required dependencies where appropriate.
5. Review the plan and create the instance.

4. Practical Example

Create main.tf:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

variable "aws_region" {
  type    = string
  default = "us-east-1"
}

variable "ami_id" {
  type        = string
  description = "Valid AMI ID in the selected region"
}

variable "subnet_id" {
  type = string
}

variable "security_group_id" {
  type = string
}

data "aws_subnet" "existing" {
  id = var.subnet_id
}

data "aws_security_group" "existing" {
  id = var.security_group_id
}

resource "aws_instance" "app" {
  ami                    = var.ami_id
  instance_type          = "t3.micro"
  subnet_id              = data.aws_subnet.existing.id
  vpc_security_group_ids = [data.aws_security_group.existing.id]

  tags = {
    Name        = "terraform-app-server"
    Environment = "dev"
    ManagedBy   = "Terraform"
  }
}

output "instance_id" {
  value = aws_instance.app.id
}

output "private_ip" {
  value = aws_instance.app.private_ip
}
```

Create terraform.tfvars with actual values:

```
aws_region        = "us-east-1"
ami_id            = "ami-REPLACE_WITH_VALID_AMI"
subnet_id         = "subnet-REPLACE_WITH_REAL_ID"
security_group_id = "sg-REPLACE_WITH_REAL_ID"
```

The IDs above are placeholders and must be replaced with valid values. The subnet and security group must belong to the same VPC, and the security group must allow only the required traffic.

Execute:

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

This project illustrates the relationship between read-only data sources and a managed EC2 resource. It does not create a VPC or security group.

5. Scenario-Based Interview Questions

Q1. Why are we using data sources instead of creating a VPC and security group?

Answer: The VPC and security group already exist and may be managed by another team. Data sources let us retrieve their information without declaring them as resources to create or destroy.

Q2. What if the subnet belongs to a different VPC than the security group?

Answer: The configuration is invalid for the intended EC2 network attachment. I would verify that both IDs belong to the same VPC before applying.

Q3. The plan is successful, but EC2 creation fails. What would you investigate?

Answer: I would check IAM permissions, subnet and security-group compatibility, AMI availability, instance quotas, subnet capacity, and the detailed AWS API error.

Topic 24: Advanced Interview Questions — Real Production Scenarios

Scenario 1: A production EC2 instance is unexpectedly marked for replacement.

Answer: I would not apply the change immediately. I would identify the attribute causing replacement, check the provider's behavior, inspect the current state and configuration, and determine the operational impact. If replacement is expected, I would plan a safe rollout with appropriate backups and availability controls.

Scenario 2: Two teams are using the same VPC. One team accidentally deletes a resource managed by the other.

Answer: I would investigate ownership, review the Terraform state and configuration, assess the impact, and recover the resource through an approved process. I would improve ownership boundaries, IAM permissions, change approvals, and shared-infrastructure practices to prevent recurrence.

Scenario 3: terraform plan shows that a resource will be destroyed after removing a block.

Answer: I would determine whether the resource is still required. If it must remain, I would restore the configuration or use an appropriate state-management operation under a controlled migration plan. I would not apply the destructive plan without understanding its consequences.

Scenario 4: Terraform creates duplicate resources after a team migration.

Answer: I would check whether the existing objects are already recorded in the correct state. I would assess whether state migration or importing is needed, verify resource addresses, and prevent concurrent changes during the recovery.

Scenario 5: The AWS Console shows a different instance type from Terraform code.

Answer: I would run a plan to identify drift, determine who changed the instance and why, and establish the intended configuration. I would then update Terraform code or restore the approved configuration, depending on the requirement.

Scenario 6: A data source fails even though the resource exists in AWS.

Answer: I would verify the provider's region and account, the lookup ID or filters, IAM permissions, and whether the selected provider configuration is correct.

Quick revision: Important Terraform commands

Command	Purpose
terraform init	Initialize the working directory and install providers
terraform fmt	Format Terraform configuration
terraform validate	Validate configuration consistency
terraform providers	Show provider requirements
terraform plan	Preview infrastructure changes
terraform apply	Execute planned changes
terraform state list	List resource addresses in state
terraform state show ADDRESS	Inspect a resource in state
terraform import ADDRESS ID	Associate a supported existing object with state
terraform plan -destroy	Preview destruction of managed infrastructure

Final study checklist

Part 4 progress: 0/12 completed

- [ ] Providers and provider version constraints
- [ ] Provider aliases and multi-region configurations
- [ ] Resources, arguments, and attributes
- [ ] Data sources and resource vs. data source
- [ ] Implicit and explicit dependencies
- [ ] Lifecycle rules and resource replacement
- [ ] count and for_each
- [ ] provider, providers, and dynamic blocks
- [ ] Provisioners and their limitations
- [ ] Drift detection and importing existing resources
- [ ] Practical AWS EC2 project
- [ ] Production scenario-based interview answers

Interview tip: For every Terraform scenario, explain three things clearly: what Terraform will do, why it will do it, and how you would verify the plan before applying changes.